

Task Tracker API — System Architecture Scope

1. Architecture Scope

The **Task Tracker API** will be a backend REST API for B2B organizations to manage users and their assigned tasks.

In Scope

- **API layer:** RESTful HTTP endpoints for client applications.
- **Authentication & authorization:** User signup, authentication, and role/permission checks.
- **User management:** Organization users and their basic account information.
- **Task management:** Create, retrieve, update, and manage task status.
- **Multi-tenant organization support:** Ensure users can access only data belonging to their organization.
- **Validation:** Validate request data and return appropriate HTTP status codes and error messages.
- **Security:** Password protection, authenticated requests, authorization, and input validation.
- **Observability:** Basic application logging and error tracking.
- **API documentation:** Document endpoints, request/response formats, and authentication requirements.

Out of Scope for This Stage

- Database schema design
 - Database technology selection
 - Backend implementation/code
 - Frontend/UI development
 - Detailed infrastructure/deployment architecture
 - Advanced reporting, analytics, notifications, or integrations
-

2. Core B2B User Stories

User Story 1 — User Signup

As a new business user,

I want to create an account associated with my organization,
so that I can securely access the Task Tracker API and manage tasks.

Acceptance Criteria

1. **Valid signup**
 - Given a user provides all required signup information,
 - When the signup request is submitted,
 - Then the API creates the user account and returns a successful response.
 2. **Required fields**
 - Given one or more required fields are missing,
 - When the signup request is submitted,
 - Then the API rejects the request with a validation error.
 3. **Duplicate account**
 - Given an account already exists for the supplied unique identifier,
 - When another signup request is submitted using that identifier,
 - Then the API rejects the request and returns an appropriate conflict/error response.
 4. **Password security**
 - Given a user supplies a password,
 - When the account is created,
 - Then the API must not expose or return the user's plaintext password.
 5. **Organization association**
 - Given the signup process requires an organization,
 - When signup succeeds,
 - Then the created user is associated with the appropriate organization.
-

User Story 2 — Task Creation

As an authenticated business user,

I want to create a task within my organization,
so that work can be assigned and tracked.

Acceptance Criteria

1. **Authenticated access**
 - Given the user is authenticated and authorized,
 - When they submit a valid task creation request,
 - Then the API creates the task.
2. **Required task information**
 - Given required task information is missing or invalid,
 - When the request is submitted,
 - Then the API rejects the request with a validation error.
3. **Organization isolation**

- Given the authenticated user belongs to Organization A,
 - When they create a task,
 - Then the task belongs to Organization A and cannot inadvertently be created under another organization.
4. **Task ownership/assignment**
 - Given a valid assignee is specified,
 - When the task is created,
 - Then the task records the appropriate assignment.
 5. **Successful response**
 - Given the task is successfully created,
 - Then the API returns an appropriate success status and sufficient task information for the client to identify the newly created task.
-

User Story 3 — Task Updates

As an authenticated business user,
I want to update a task that I am authorized to modify,
so that its details and progress remain current.

Acceptance Criteria

1. **Authenticated and authorized update**
 - Given the user is authenticated and has permission to modify the task,
 - When they submit valid changes,
 - Then the API updates the task successfully.
2. **Unauthorized update**
 - Given the user does not have permission to modify a task,
 - When they attempt to update it,
 - Then the API rejects the request without modifying the task.
3. **Task existence**
 - Given the specified task does not exist,
 - When an update is attempted,
 - Then the API returns an appropriate not-found response.
4. **Valid status transitions**
 - Given a task status is being changed,
 - When the requested status is valid,
 - Then the API applies the status change.
5. **Invalid data**
 - Given the update contains invalid field values,
 - When the request is submitted,
 - Then the API rejects the update with a validation error and does not apply the invalid changes.
6. **Organization isolation**
 - Given the task belongs to a different organization from the authenticated user,
 - When the user attempts to update it,

- Then the API denies access and leaves the task unchanged.

CODE CONTROLLER

```
CREATE TABLE users (  
  id BIGINT PRIMARY KEY,  
  name VARCHAR(255) NOT NULL,  
  email VARCHAR(255) NOT NULL UNIQUE,  
  password_hash VARCHAR(255) NOT NULL,  
  created_at TIMESTAMP NOT NULL  
);
```

```
CREATE TABLE tasks (  
  id BIGINT PRIMARY KEY,  
  user_id BIGINT NOT NULL,  
  title VARCHAR(255) NOT NULL,  
  description TEXT,  
  status VARCHAR(50) NOT NULL,  
  due_date TIMESTAMP,  
  created_at TIMESTAMP NOT NULL,
```

```
  CONSTRAINT fk_tasks_user  
    FOREIGN KEY (user_id)  
    REFERENCES users(id)  
);
```